

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

MÁSTER UNIVERSITARIO EN CIBERSEGURIDAD

Análisis de código en el lenguaje C con LLMs buscando errores de seguridad causados por heap y por stack

Static Code Analysis of C Code with LLMs on search of security errors caused by heap and stack

Realizado por

Pablo Jesús Moreno Polo

Tutorizado por

David Santo Orcero

Departamento

Lenguajes y ciencias de la computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, septiembre de 2026

Abstract

Vulnerabilities reported in applications and services have been on an increasing trend in recent years. In C and C++, these vulnerabilities arise because they are considered programming languages with unsafe memory management. A specific case within these languages involves security errors caused by heap and call stack management.

Static code analysis tools, or SAST, are established as general-purpose tools for code analysis and vulnerability detection. Their adoption in the software development lifecycle is considered a standard. Advances in artificial intelligence in recent years have given rise to multiple large language models, with the ability to learn and retain the context of the texts they are trained on. In this project, we study how large language models compare to SAST in their effectiveness at detecting security errors in C code.

Keywords: Static Application Security Testing, Large Language Models, C programming language, False Positives, False Negatives

Resumen

Las vulnerabilidades reportadas en aplicaciones y servicios están en tendencia creciente en los últimos años. En los lenguajes C y C++ estas vulnerabilidades se dan al considerarse lenguajes de programación con una gestión insegura de la memoria. Una casuística de estos lenguajes son los errores de seguridad causados por la gestión del montículo y de la pila de llamadas.

Las herramientas de análisis de código estático, o SAST, se establecen como herramientas de uso general para el análisis de código y detección de vulnerabilidades. Su adopción en el ciclo de desarrollo de software se considera un estándar. Los avances en inteligencia artificial de los últimos años dieron lugar a múltiples modelos extensos de lenguaje, con capacidades para aprender y retener el contexto de los textos con los que se entrenan. En este proyecto estudiamos cómo se comparan los modelos extensos de lenguaje respecto a los SAST en su eficacia para detectar errores de seguridad en código en el lenguaje C.

Keywords: Analizador de código estático, Modelo extenso de lenguaje, Lenguaje de programación C, Falsos positivos, Falsos negativos

Índice

1. Introducción	9
2. Estado del arte	11
2.1. El funcionamiento de la pila de llamadas	11
2.2. El funcionamiento del montículo (<i>heap</i>)	11
2.3. Organización de la memoria interna de un proceso	12
2.4. CWEs de la pila de llamadas	13
2.5. CWEs del montículo	14
2.6. CWEs que afectan tanto a la pila de llamadas como al montículo	15
2.7. Modelo extenso de lenguaje (<i>Large Language Model, LLM</i>)	16
2.8. Transformador (<i>Transformer</i>)	17
2.9. Ajuste fino eficiente en parámetros (PEFT)	17
2.9.1. Adaptación de bajo Rango (LoRA)	18
2.10. Cuantización de modelos	19
3. Metodología de análisis	21
3.1. Contextualización	21
3.2. Conjunto de datos empleado	22
4. Herramientas empleadas	25
4.1. Python	25
4.2. Hugging Face y Hugging Face Hub	25
5. Resultados de análisis con Splint	27
5.1. Exposición de la tabla-matriz de confusión	27
5.2. Análisis de los resultados	29
5.2.1. Proceso de reporte de vulnerabilidades con Splint	29
6. Análisis con modelos extensos de lenguaje	31
6.1. Resultados de la validación de google/gemma-2-2b-it	32

6.2. Resultados de la validación de microsoft/Phi-3.5-mini-instruct	34
6.3. Procedimiento de ajuste fino de parámetros	36
6.4. Ajuste fino de parámetros y validación del modelo tiiuae/Falcon3-1B-Instruct	36
6.5. Ajuste fino de parámetros y validación del modelo meta-llama/Llama-3.2-1B-Instruct	39
6.6. Ajuste fino de parámetros y validación del modelo HuggingFaceTB/SmolLM2-1.7B-Instruct	41
7. Conclusiones	43
7.1. Lineas futuras	44

Índice de figuras

1.	Estructura de la memoria de un proceso C/C++.	12
2.	Resultados obtenidos de análisis de <i>CWE</i> con la herramienta Splint	28
3.	Resultados análisis del modelo google/gemma-2-2b-it	33
4.	Resultados análisis del modelo microsoft/Phi-3.5-mini-instruct	35
5.	Resultados análisis del modelo tiiaue/Falcon3-1B-Instruct	38
6.	Resultados análisis del modelo meta-llama/Llama-3.2-1B-Instruct . . .	40
7.	Resultados análisis del modelo HuggingFaceTB/SmolLM2-1.7B-Instruct .	42

1

Introducción

El lenguaje de programación C es uno de los lenguajes más populares en la industria de desarrollo de software. Este lenguaje adquiere una notable presencia en sistemas en los que se necesita acceso a bajo nivel al hardware, la eficiencia temporal y el control explícito de los recursos. Originalmente, C se planteó como un lenguaje de programación de sistemas para la familia de sistemas operativos tipo UNIX, caracterizado técnicamente como un lenguaje liviano, de propósito general y portable [1]. Una gran cantidad de aplicaciones de alto rendimiento y sistemas críticos se implementan en C y continúan en soporte.

Sin embargo, las características que popularizaron a C también son el fondo detrás de una gestión insegura de la memoria intrínseca al lenguaje. Esta gestión insegura propicia la aparición de errores en el manejo de la memoria, que, además de ser difíciles de detectar, constituyen una de las superficies de ataque más persistentes en los lenguajes de programación *memory-unsafe*.

Los lenguajes de programación *memory-unsafe* no cuentan, por defecto, con mecanismos de prevención de errores comunes de programación relacionados con la libertad en el acceso a memoria. Bajo esta libertad, el programador gestiona manualmente la memoria, pero esta libertad conlleva la aparición de distintos tipos de vulnerabilidades, entre las que figuran los desbordamientos de pila, las filtraciones de memoria y la corrupción de memoria.

Concretamente, las vulnerabilidades de corrupción de memoria constituyen el tipo de vulnerabilidad más frecuente. La sobre-escritura de metadatos del montículo, los punteros colgantes, las lecturas de memoria no inicializada, las liberaciones inválidas de memoria o la doble liberación de memoria son debilidades reconocidas por investigadores de seguridad y cuentan con sus respectivas técnicas de protección para abordarlas [2].

Cada una de las vulnerabilidades anteriormente mencionadas se recogen en la Common Weakness Enumeration (CWE), por lo que hacemos énfasis en que las fallas que se encuentran

en programas escritos en C no son errores individuales aislados, sino que son consecuencia del diseño estructural del lenguaje.

A la vista de este escenario, la detección de debilidades supone una necesidad con objeto de asegurar la calidad y diseñar software seguro. La literatura sobre el análisis de programas señala una reducción en el coste de corrección de vulnerabilidades si se detectan tempranamente. De hecho, facilita la implementación de prácticas de seguridad «*shift-left*» e intervenir antes de que los defectos lleguen a la etapa de despliegue o de explotación [3].

Los analizadores de código estático son herramientas que permiten la detección de vulnerabilidades en el código fuente a partir de la detección de patrones. Son herramientas de referencia en la industria de desarrollo software, por ejemplo en las canalizaciones de integración continua, dado que permiten identificar estas vulnerabilidades en las fases tempranas del ciclo de vida del software. Por el contrario, hay estudios empíricos que muestran sus limitaciones en la precisión, cobertura, explicabilidad y unas altas tasas de positivos en sus reportes [4]. Por lo tanto, detectar estas vulnerabilidades asegurando una alta fiabilidad y utilidad sigue siendo un problema abierto.

Estado del arte

En el contexto de este trabajo profundizaremos en el análisis de vulnerabilidades relacionadas con la pila de llamadas, también llamada pila o *call stack* y el montículo, también llamado *heap*, en el lenguaje C. Ambas son dos regiones de memoria relevantes en la estructura de los programas en tiempo de ejecución. Expondremos el funcionamiento de ambas, y posteriormente explicaremos sus debilidades.

2.1. El funcionamiento de la pila de llamadas

En la pila de llamadas (*call stack*) se almacenan los datos relativos a las subrutinas de un programa. Cuando se invoca una subrutina, se crea en la pila un nuevo marco *stack frame*, que contiene sus parámetros, la dirección de retorno y sus variables locales. Al finalizar la subrutina, su marco se libera.

La pila sigue una estrategia LIFO (*last in, first out*): el último marco en crearse es el primero en destruirse. Por ello, las subrutinas más anidadas (las llamadas más recientes) son las últimas en asignarse pero las primeras en liberarse, de forma análoga a una pila de libros donde solo se puede retirar el libro de la cima.

La gestión de la pila se realiza mediante un puntero de pila. Al insertar un nuevo marco, el puntero se desplaza hasta el tope de dicho marco. Y al salir, el puntero retrocede hasta el tope del marco anterior (o al límite inicial si la pila queda vacía). Estos movimientos son automáticos y el programador no tiene control directo sobre ellos.

2.2. El funcionamiento del montículo (*heap*)

El montículo (*heap*) es una región de memoria dinámica; es decir, los datos almacenados en él se asignan durante el tiempo de ejecución del programa. A diferencia de la pila, los datos del

montículo deben gestionarse manualmente.

La gestión manual de la memoria comprende dos operaciones fundamentales: la petición y la liberación de bloques de memoria. Para solicitar un bloque de un número específico de bytes, se emplean las funciones `malloc()` o `calloc()`, que devuelven la dirección de memoria del bloque asignado. La liberación se realiza mediante la función `free()`, a la que se proporciona como argumento la dirección del bloque que se desea liberar.

Al delegar la gestión de la memoria en el programador, los errores aparecen con una frecuencia mucho mayor. Uno de los errores más característicos son las **fugas de memoria** (memory leaks), que ocurren cuando no se libera un bloque de memoria tras cumplir su utilidad. En casos severos, puede llevar al agotamiento de los recursos (especialmente en dispositivos con capacidades limitadas), lo que puede derivar en una denegación de servicio [5].

2.3. Organización de la memoria interna de un proceso

Dentro de un proceso, la pila y el montículo comparten la misma región de memoria, con la particularidad de que ambas crecen en direcciones opuestas. Por un lado, la pila tiene un límite de tamaño impuesto por el sistema: en Linux puede crecer hasta 8 MB, mientras que en Windows el límite típico es de 1 MB. En cambio, el montículo puede crecer hasta agotar la memoria virtual disponible. En la figura 1 se observa esta región compartida dentro de la estructura de memoria de un proceso.

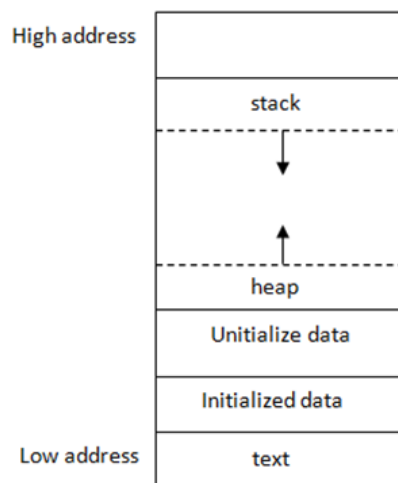


Figura 1: Estructura de la memoria de un proceso C/C++. Extraída de [6]

2.4. CWEs de la pila de llamadas

Las debilidades que vamos a tratar se recogen en el catálogo CWE (*Common Weakness Enumeration*) de MITRE, el cual contiene fallos estandarizados conocidos por causar vulnerabilidades. Tanto la gestión de la pila, como la región de memoria en sí, tienen asociadas múltiples debilidades, de forma que en este artículo recogemos las ocho más relevantes para el propósito de nuestro estudio.

- **CWE-121: Desbordamiento del búfer en la pila.** Ocurre cuando se escribe más allá de los límites de un búfer asignado en la pila. Pudiendo corromper datos en marcos adyacentes, como variables o direcciones de retorno, y potencialmente permitir la ejecución de código arbitrario [7].
- **CWE-562: Devolución de puntero de pila.** Ocurre cuando una función devuelve un puntero a una variable local. Al finalizar la ejecución de dicha función, su marco de pila se destruye y todas las referencias a las variables locales que contenía quedan invalidadas. Por tanto, el puntero devuelto apunta a una región de memoria ya desasignada, cuyo uso posterior constituye un comportamiento indefinido y potencialmente explotable [8].
- **CWE-628: Llamada a función con argumentos especificados incorrectamente.** Constituye una categoría base de vulnerabilidades de la cual se desglosan varias vulnerabilidades hijas. Entre las vulnerabilidades hijas podemos encontrar tres específicas de la pila de llamadas.

CWE-683: Llamada a función con argumentos en orden incorrecto.

CWE-685: Llamada a función con número incorrecto de argumentos.

CWE-688: Llamada a función con una variable o referencia incorrecta como argumento.

- **CWE-674: Recursión no controlada.** Se da en el caso de una función recursiva la cual carece de un caso base adecuado o de un límite en su profundidad, lo que lleva a un consumo masivo de memoria de pila (*stack memory*) y al agotamiento de su espacio.
- **CWE-835: Bucle con condición de salida inalcanzable.** Se da cuando el programa entra

en un bucle infinito, lo que lleva a un agotamiento de la pila. Un ejemplo puede ser un bucle `while(true)` sin una instrucción `break`.

2.5. CWEs del montículo

Hay muchos CWEs recogidos que pueden asociarse al montículo, de forma que para nuestro estudio hemos realizado un listado con sus ocho debilidades más relevantes.

- **CWE-122: Desbordamiento del búfer en el montículo.** Ocurre cuando en un bloque de memoria se escriben más datos de los que caben, de forma que compromete la memoria adyacente y puede derivar en la ejecución de código arbitrario.
- **CWE-244: Limpieza incorrecta de la memoria antes de su liberación (*Heap Inspection*).** Ocurre cuando se libera un bloque de memoria (ya sea explícitamente con `free()` o implícitamente con `realloc()`) sin sobrescribir previamente los datos sensibles que contenía. En particular, cuando `realloc()` necesita reubicar el bloque en una nueva dirección copia su contenido, libera el bloque original y lo deja intacto en el *heap*, aunque inaccesible para el programa. Un atacante que pueda leer volcados de memoria o direcciones liberadas podría recuperar dichos datos [9].
- **CWE-401: Filtraciones de memoria (*Memory Leaks*).** Ocurre cuando un bloque de memoria no es liberado después de haber cumplido su función, lo que provoca su acumulación y la imposibilidad de reutilizarlo para otros fines. Cuando el problema persiste de manera sistémica, pueden producirse acumulaciones masivas de memoria. En este escenario, un atacante podría explotar esta debilidad para causar una denegación de servicio (*DoS*, en inglés *Denial of Service*) [10].
- **CWE-415: Doble liberación.** Se da cuando se llama a la función `free()` dos veces sobre la misma dirección de memoria. Esto corrompe la estructura del montículo y puede derivar en fallos inesperados o comportamiento impredecible; en escenarios controlados por un atacante, puede posibilitar ataques de uso tras liberación (*use-after-free*), que a su vez permiten ejecutar código arbitrario o escalar privilegios [11].

CWE-416: Uso tras la liberación (*Use-After-Free*). Ocurre cuando se accede a una región de memoria tras haber sido liberada, sin que el puntero haya sido reasignado o

puesto a NULL. Esto provoca que un atacante pueda reutilizar un bloque de memoria liberado, lo que le permite llevar a cabo los ataques previamente descritos [12].

- **CWE-476: Desreferencia de puntero nulo.** Ocurre cuando se emplea un puntero bajo la expectativa de que contenga una referencia a una dirección de memoria válida; no obstante, el puntero contiene una referencia a NULL, lo que provoca un fallo en el proceso. En circunstancias excepcionales, cuando NULL apunta a la dirección de memoria 0x0 y el código privilegiado puede acceder a ella, de forma que es posible leer o escribir en dicha memoria, puede derivar en la ejecución de código arbitrario [13].
- **CWE-590: Liberación de memoria no asignada en el montículo.** Ocurre cuando se llama a la función `free()` con un puntero de memoria no asignado en el montículo. Al liberar un puntero inválido, puede corromperse el manejo de las estructuras de datos de memoria, lo que provoca fallos en el programa. En el caso de un ataque, un atacante puede usar la función `free()` para realizar operaciones en memoria de forma que le permitan modificar controles críticos o ejecutar código arbitrario [14].
- **CWE-761: Liberación de puntero no situado al inicio del búfer.** Ocurre cuando se llama a la función `free()` con una dirección de memoria asignada en el montículo, pero que no está al inicio del búfer, lo que provoca un fallo en el programa [15].

2.6. CWEs que afectan tanto a la pila de llamadas como al montículo

Las vulnerabilidades recogidas en este apartado se encuentran asociadas tanto a la pila de llamadas como al montículo, por lo que les hemos dedicado un apartado propio. Estas vulnerabilidades comparten una debilidad común, el desbordamiento de búfer, de forma que aprovechan las características de este tipo de vulnerabilidad para explotarla por otros métodos, como veremos a continuación. Recogemos un total de cinco vulnerabilidades más.

- **CWE-123: (*Write-what-where*).** Esta vulnerabilidad consiste en cualquier condición que permite a un atacante escribir un valor arbitrario en una dirección de memoria también arbitraria. Este escenario es frecuente en el caso de los desbordamientos de búfer, ya que, al escribir datos que exceden el tamaño del búfer reservado, se pueden sobrescribir

posiciones de memoria, lo que permite la ejecución de código arbitrario o la subversión de las medidas de seguridad [16].

- **CWE-125: Lectura fuera de los límites del búfer.** Ocurre cuando se realiza una lectura de los datos fuera de las direcciones del búfer. De esta debilidad se desglosan otras dos vulnerabilidades hijas:

CWE-126: Lectura de datos más allá del búfer

CWE-127: Lectura de datos atrás del búfer

- **CWE-680: Desbordamiento de entero que conduce a desbordamiento de búfer.** Esta vulnerabilidad ocurre cuando, al calcular el tamaño de un búfer, el resultado de una operación aritmética supera el valor máximo representable por el tipo entero utilizado. En consecuencia, el búfer asignado tiene una capacidad menor a la esperada, lo que provoca un desbordamiento de búfer al intentar escribir en él más datos de los que puede albergar [17].

2.7. Modelo extenso de lenguaje (*Large Language Model, LLM*)

Un modelo extenso de lenguaje es un tipo de modelo de aprendizaje profundo (*deep learning*). En concreto, se trata de un sistema de inteligencia artificial basado en redes neuronales de múltiples capas. Dichos algoritmos emplean cantidades masivas de parámetros y extensos conjuntos de datos de entrenamiento para llevar a cabo tareas de comprensión y predicción de texto [18].

Por consiguiente, estos modelos se inscriben en el ámbito del procesamiento del lenguaje natural (*NLP*, del inglés *Natural Language Processing*). La función principal que desempeñan es la predicción estocástica del siguiente tóken en una secuencia. Este comportamiento se consigue durante el entrenamiento: a partir de los textos con los que se entrenan, aprenden distribuciones de probabilidad sobre secuencias de tókenes, lo que les permite generar texto que sigue los patrones estadísticos observados en los datos. [19]

La aplicación de esta funcionalidad se extiende más allá del lenguaje natural, abarcando también el código fuente de programas informáticos. Estos modelos resultan útiles en tareas como la comprensión, generación o explicación de código, la creación de casos de prueba uni-

tarios y la generación de documentación técnica.

2.8. Transformador (*Transformer*)

El transformador es una arquitectura basada en redes neuronales artificiales con capacidad para procesar secuencias de datos, como textos, imágenes o vídeos. El mecanismo introducido en esta arquitectura, que resulta novedoso, es el mecanismo de auto-atención, en inglés, *Self-Attention Mechanism*.

El mecanismo de auto-atención consiste en la medición de relevancia de cada palabra o elemento respecto al resto dentro de una misma secuencia. Esta medición se representa por medio de tres vectores: Q, K y V. Donde Q representa la consulta de la palabra en el contexto, K representa el término sobre el cual se realiza la consulta y V la información semántica asociada a cada clave.

Cada uno de los valores de Q, K y V se extrae de sus respectivas matrices de pesos W_q , W_k y W_v . De forma que el cómputo de cada uno de estos vectores se formaliza con las operaciones.

- $Q = W_q \cdot x$
- $K = W_k \cdot x$
- $V = W_v \cdot x$

Donde x es un *embedding* de palabras, es decir, una representación del texto en un arreglo numérico.

En la arquitectura del transformador se distinguen dos partes: un codificador y un decodificador. El codificador recibe la entrada y el decodificador genera la salida. El diseño original de esta arquitectura lo propusieron ingenieros de Google para la traducción automática de textos [20]. Sin embargo, esta arquitectura se empleó para otros propósitos, el más singular de estos es su uso en modelos de inteligencia artificial generativa, entre los que se incluyen los *LLM*, por destacar en tareas de «comprensión» del lenguaje.

2.9. Ajuste fino eficiente en parámetros (PEFT)

También se le denomina como *fine-tuning* eficiente en parámetros. Es una metodología empleada para mejorar el rendimiento de modelos de inteligencia artificial previamente entrenados.

Las mejoras de rendimiento pueden consistir en que el modelo ejecute tareas que previamente no podía realizar o que aprenda conjuntos de datos específicos [21].

El ajuste eficiente consiste en un re-entrenamiento de un subconjunto de los parámetros del modelo, de forma que el resto de la estructura del modelo se conserva. La preferencia de esta metodología en lugar de un *fine-tuning* tradicional se vincula a las siguientes circunstancias:

Prevención del «olvido catastrófico», que es la pérdida del contexto adquirido en fases de entrenamiento previas, debido a un reajuste de todos los parámetros del modelo durante el proceso de re-entrenamiento o ajuste.

Reducción del riesgo de sobreajuste. El cual es sustancialmente mayor si se induce el modelo a un re-entrenamiento completo, provocando un olvido del contexto con el que se entrenó previamente, puesto que se re-ajustan todos los parámetros nuevamente.

Disminución en la demanda de datos. Conforme más parámetros se entrenan, más cantidades de datos se necesitan para disponer al modelo del contexto suficiente para ejecutar la tarea para la cual se entrena.

2.9.1. Adaptación de bajo Rango (LoRA)

La adaptación de bajo rango es una técnica empleada para la adaptación de un modelo de inteligencia artificial a nuevos contextos. Evita el re-entrenamiento de potencialmente miles de millones de parámetros, o al menos medio billón de parámetros, agregando matrices de rango bajo que, al aplicarlas a las entradas, permiten ajustar el modelo para obtener resultados específicos según el contexto [22].

Estas matrices de rango bajo son las que se ajustan en el proceso de entrenamiento, mientras que el resto de pesos y parámetros originales del modelo se congelan. Por lo tanto, se reduce la carga en memoria y en tiempo de computación a la que se somete el modelo.

Las matrices de rango bajo son de dimensiones inferiores a las matrices de rango superior que conforman el modelo, se emplean de manera que al agregarlas a las ponderaciones del modelo base se ajusta al contexto deseado, resultando efectivamente en agregados de mucho menor tamaño relativo al modelo.

2.10. Cuantización de modelos

La cuantización es un proceso de compresión y optimización de los parámetros de un modelo de inteligencia artificial. Este proceso consiste en la conversión de valores de alta precisión en valores con menor precisión: por ejemplo de valores de punto flotante en 32 bits (denominados como FP32) a valores enteros en 8 bits (expresados como INT8).

Este proceso conlleva el denominado error de cuantización, consecuencia de la pérdida de información al expresar la información en variables menos «enriquecidas». Sin embargo, al aplicar esta cuantización a los pesos del modelo y a sus valores de activación, se consigue una aceleración en el tiempo de cómputo y en un menor consumo de memoria.

Ciertamente, existen diversas estrategias de cuantización con las cuales se puede optar por perder más, menos o ninguna información en según qué capas o fases se consideren más críticas de un modelo. Igualmente, una cuantización «agresiva» sobre la que se controle el error de cuantización, puede lograr resultados satisfactorios con modelos muy pesados en dispositivos con menos recursos. [23]

Metodología de análisis

3.1. Contextualización

La metodología de análisis que empleamos se basa en el análisis comparativo. Su objetivo es detectar debilidades en el código fuente. Con tal fin, recopilamos los resultados del análisis de vulnerabilidades empleando dos tipologías de herramientas: los analizadores de código estático (SAST, del inglés *Static Application Security Testing*) y los modelos de lenguaje de gran tamaño (LLM, del inglés *Large Language Model*).

La detección de estas vulnerabilidades la realizamos sobre código fuente escrito exclusivamente en el lenguaje de programación C. Para tal fin, escogimos **Splint**, una herramienta estándar para el análisis estático de programas en C. Existen diversos tipos de herramientas de análisis estático, entre las cuales se encuentran los *linter*. Según la definición de Sonar, los *linter* analizan el código estático para identificar y señalar problemas que pueden derivar en errores, vulnerabilidades y *code smells* [24]. En esencia, estas herramientas buscan patrones en el código que correspondan a errores conocidos, defectos, problemas de estilo o de arquitectura. Splint es una herramienta que pertenece a esta categoría.

La principal debilidad de los analizadores de código estático es su elevada tasa de falsos positivos. Esto se debe a que estas herramientas operan con una carencia de contexto, dado que disponen de una cantidad muy limitada de información sobre el código analizado. La detección de vulnerabilidades y errores se basa en reglas heurísticas. Cuando dichas reglas son ambiguas, múltiples patrones se señalan como positivos aún cuando el código es válido o la alerta es incorrecta. Por otro lado, si las reglas son demasiado estrictas, pueden generar falsos positivos que no se corresponden con la vulnerabilidad central, sin ofrecer una solución al problema de

raíz [25].

En contraste, los *LLM* no presentan esa carencia en contexto, dado que fueron entrenados con conjuntos de datos masivos, lo que les permite aprender patrones, estructuras y comportamientos tanto de vulnerabilidades comunes como nuevas [26]. Estudios recientes demuestran que determinados modelos logran una alta precisión en la clasificación de código vulnerable, superando a otras arquitecturas en esta métrica. Sin embargo, en el contexto actual no se plantean como sustitutas, sino como integraciones dentro de *SAST* de forma que mejoran su rendimiento en escenarios específicos [27].

Para evaluar el rendimiento de los modelos de lenguaje en detección de vulnerabilidades, seleccionaremos cinco modelos ampliamente utilizados en el dominio de la programación. A continuación, analizaremos los casos de prueba pertenecientes a un conjunto de datos previamente seleccionado, utilizando tanto la herramienta Splint como los modelos mencionados. Finalmente, se comparan los resultados obtenidos con el fin de determinar si, en este contexto, los LLM superan, igualan o presentan un rendimiento inferior al de Splint.

3.2. Conjunto de datos empleado

El conjunto de datos que empleamos es la suite de pruebas Juliet. Es un conjunto, desarrollado por el Centro de Software Confiable (CAS, del inglés *Center for Assured Software*) de la NSA (*National Security Agency*), el cual contiene referencias para la evaluación de herramientas de análisis estático de seguridad.

En todos los casos de prueba se distinguen tres secciones.

1. Una sección *bad*. La cual contiene código vulnerable que presenta un CWE específico.
2. Una sección *good*. La cual contiene código seguro. Este código es el código vulnerable al cual se le incorpora un *fix* para solucionar la debilidad del caso vulnerable.
3. Una sección *main*. Que permite la ejecución tanto del código vulnerable como del código seguro.

La ejecución tanto del caso vulnerable como del caso seguro puede controlarse según dos macros OMITGOOD y OMITBAD. Según cuál se defina se omitirá la sección segura o la vulnerable, respectivamente.

Para nuestro análisis seleccionamos la suite Juliet C/C++ en su versión 1.3, la cual recoge más de 64.000 casos de prueba para 118 *CWE* distintos [28]. Dentro del *corpus* de pruebas escogimos únicamente aquellos casos en C y relativos a las debilidades expuestas desde el apartado 2.4 al apartado 2.6 de este trabajo.

4

Herramientas empleadas

4.1. Python

Python es un lenguaje de programación que cuenta con una comunidad de gran tamaño en el campo de la ciencia de los datos y la inteligencia artificial. Su sintaxis sencilla, sumada a su enorme ecosistema de desarrollo conformado por múltiples librerías especializadas para labores de inteligencia artificial y análisis de datos, p.ej., PyTorch, Scikit-learn o Pandas; las cuales cuentan con apoyo continuo por parte de la comunidad de desarrolladores, lo hacen una opción relevante en cuanto a la realización de labores de investigación y desarrollo.

4.2. Hugging Face y Hugging Face Hub

Hugging Face es un proveedor de herramientas de desarrollo de aplicaciones de aprendizaje automático. La empresa ofrece paquetes con herramientas para el diseño y ajuste de modelos de inteligencia artificial. Sus paquetes reciben actualizaciones constantemente y cuentan con todas las herramientas a las que se hizo referencia en la recopilación del estado del arte actual. Por ello, tomamos su tecnología como referencia para nuestros estudios.

Por otro lado, la organización Hugging Face también administra la plataforma online Hugging Face Hub, la cual recoge publicaciones, tanto de particulares como de organizaciones, de modelos de inteligencia artificial y conjuntos de datos de entrenamiento. Escogimos esta plataforma porque es la que contiene la colección de modelos de inteligencia artificial de acceso abierto (*open source*) más actualizada de Internet. Esto es especialmente relevante dado que permite el acceso a los modelos de inteligencia artificial más avanzados desarrollados por las principales organizaciones dedicadas a la investigación y desarrollo en esta materia [29].

Adicionalmente, la plataforma cuenta con un sistema de reputación de los artefactos publicados basado en mecanismos de retroalimentación cuantificables: *likes* y número de descargas realizadas en el último mes.

Resultados de análisis con Splint

5.1. Exposición de la tabla-matriz de confusión

En la figura 2 exponemos los resultados obtenidos. Se expone en una tabla en la cual se presenta la matriz de confusión de cada uno de los tipos de vulnerabilidades en formato fila. A continuación se expone una leyenda del significado de cada una de las columnas de la matriz de confusión.

- Verdadero positivo (VP). Es el análisis de un caso *bad* para el cual Splint **advierde** que existe la vulnerabilidad en concreto.
- Verdadero negativo (VN). Es el análisis de un caso *good* para el cual Splint **no advierde** que existe la vulnerabilidad en concreto.
- Falso positivo (FP). Es el análisis de un caso *good* para el cual Splint **advierde** que existe la vulnerabilidad en concreto.
- Falso negativo (FN). Es el análisis de un caso *bad* para el cual Splint **no advierde** que existe la vulnerabilidad en concreto.

Debilidad	Nº casos de prueba	Nº de pruebas realizadas	VP	VN	FP	FN
CWE-121	5906	11812	4183	1723	4183	1723
CWE-122	3656	7312	2158	1498	2158	1498
CWE-123	168	336	150	18	150	18
CWE-124	1896	3792	1347	549	1347	549
CWE-126	1380	2760	945	435	945	435
CWE-127	1896	3792	1222	674	1222	674
CWE-401	1228	2456	600	628	600	628
CWE-415	336	672	246	90	246	90
CWE-416	150	300	132	18	132	18
CWE-476	372	744	238	134	238	134
CWE-562	2	4	0	2	0	2
CWE-674	2	4	0	2	0	2
CWE-680	336	672	117	219	117	219
CWE-685	18	36	18	0	18	0
CWE-688	18	36	18	0	18	0
CWE-761	672	1344	536	136	536	136
CWE-835	6	12	0	6	0	6
Total de casos de prueba: 18.192 (pruebas realizadas: 36.384)						

Figura 2: Resultados obtenidos de análisis de *CWE* con la herramienta Splint

5.2. Análisis de los resultados

Se observa un patrón recurrente en todos los casos: los falsos positivos coinciden con los verdaderos positivos, mientras que los falsos negativos coinciden con los verdaderos negativos. Ello se debe a que, cuando la herramienta detecta una vulnerabilidad en el código, la identifica tanto en la prueba que contiene únicamente código vulnerable como en aquella que presenta código seguro. Esta situación es consecuencia de que las modificaciones introducidas en los casos de prueba para tornar el código seguro consisten en cambios de una sola línea o en la modificación de una asignación de variable. Por consiguiente, Splint genera una cantidad considerable de falsos positivos en contextos en los que, propiamente, no existen vulnerabilidades.

5.2.1. Proceso de reporte de vulnerabilidades con Splint

En el proceso de reporte de vulnerabilidades mediante la herramienta Splint, analizamos las salidas en busca de palabras clave (*keywords*) que permitieran identificar fallos de seguridad. Cabe resaltar que Splint no posee la capacidad de detectar debilidades específicas, sino que es una herramienta especializada en la identificación de problemas como desbordamientos de búfer, usos de memoria no inicializada, pérdidas de memoria y violaciones de tipos.

En consecuencia, estos indicios no evidencian de manera concluyente la existencia de ciertas vulnerabilidades. Por ejemplo, la vulnerabilidad clasificada como CWE-123, que consiste en la escritura de valores arbitrarios en direcciones de memoria también arbitrarias, no se corresponde directamente con ninguna de las categorías de errores que Splint detecta de forma explícita. Si bien es cierto que algunas de las advertencias generadas por Splint pueden proporcionar indicios de este tipo de vulnerabilidades, dichas evidencias no resultan concluyentes, dado que la herramienta se centra en la detección de errores y fallas en el código, y no en la identificación específica de debilidades según la clasificación CWE.

6

Análisis con modelos extensos de lenguaje

Inicialmente realizamos una búsqueda de modelos extensos de lenguaje publicados que previamente contasen con un ajuste fino de parámetros para la tarea de clasificar la vulnerabilidad encontrada en código fuente en el lenguaje C.

Para el diseño del conjunto de datos de entrenamiento y de validación partimos de un conjunto de datos de Hugging Face en el cual se recopilan todos los casos de prueba de la suite de pruebas Juliet C/C++ en su versión 1.3 [30].

Para comparar el rendimiento de los modelos extensos de lenguaje con Splint, elegimos cinco modelos de inteligencia artificial sobre los que previamente aplicamos una cuantización de 4 bits y realizamos las siguientes tareas.

- `google/gemma-2-2b-it` [31] [32]. Se realizó una validación del modelo.
- `microsoft/Phi-3.5-mini-instruct` [33] [34]. Se realizó una validación del modelo.
- `meta-llama/Llama-3.2-1B-Instruct`. Se realizó un ajuste fino de parámetros y una posterior validación del modelo ajustado.
- `tiiuae/Falcon3-1B-Instruct`. Se realizó un ajuste fino de parámetros y una posterior validación del modelo ajustado.
- `HuggingFaceTB/SmolLM2-1.7B-Instruct`. Se realizó un ajuste fino de parámetros y una posterior validación del modelo ajustado.

Para validar tanto el modelo `google/gemma-2-2b-it` como el modelo `microsoft/Phi-3.5-mini-instruct` utilizamos un conjunto de datos conformado por un total de 15.000 casos.

El resto de modelos se entrenaron con un conjunto de datos de entrenamiento de 15.000 casos, y para la validación empleamos un conjunto de datos de validación de 5.000 casos. De forma que la proporción de casos de prueba respecto a casos de validación de 75/25.

6.1. Resultados de la validación de google/gemma-2-2b-it

El informe con los resultados de validación del modelo se encuentra detallado en la figura 3.

Los resultados revelan que la precisión global del modelo es de un 24 %, indicando un rendimiento bastante pobre. Este resultado es esperable dado que el modelo Gemma 2 no se entrenó para este propósito.

Gemma 2 es un modelo conversacional diseñado para el resumen de textos, especialmente de documentos extensos, y de grandes volúmenes de informes o de trabajos de investigación [35]. Los resultados confirman este hecho: el rendimiento del modelo es muy pobre en esta tarea porque no fue entrenado para el propósito de reconocer debilidades en muestras de código.

Realizando un análisis de los resultados del informe, observamos que el modelo marcó una gran cantidad de falsos positivos, especialmente de clases con las cuales no contábamos con soporte en nuestro conjunto de datos. Entre los resultados contamos con las debilidades CWE-79, CWE-89, CWE-129, CWE-131, CWE-135, CWE-193 y CWE-244.

El ejemplo más claro de falso positivo, del cual además no se contaba con ninguna muestra de soporte, es el CWE-79. Un total de 5641 muestras se marcaron como positivas de esta debilidad cuando el soporte de esta vulnerabilidad en el conjunto de datos de validación es nulo.

Resumidamente, afirmamos que el rendimiento del modelo es pésimo para la tarea encomendada, teniendo un rendimiento significativamente peor que Splint.

Debilidad	Precisión	Exhaustividad	Puntuación F1	Soporte
CWE-121	0.42	0.55	0.48	1956
CWE-122	0.81	0.27	0.41	1286
CWE-123	1.00	0.12	0.22	8
CWE-124	0.58	0.69	0.63	729
CWE-126	0.51	0.69	0.59	543
CWE-127	0.70	0.70	0.70	777
CWE-129	0.00	0.00	0.00	0
CWE-131	0.00	0.00	0.00	0
CWE-135	0.00	0.00	0.00	0
CWE-193	0.00	0.00	0.00	0
CWE-244	0.00	0.00	0.00	0
CWE-401	1.00	0.06	0.11	837
CWE-415	0.54	0.35	0.42	254
CWE-416	0.14	0.38	0.20	29
CWE-476	0.00	0.00	0.00	347
CWE-562	0.00	0.00	0.00	2
CWE-590	0.74	0.71	0.72	486
CWE-674	0.00	0.00	0.00	2
CWE-680	1.00	0.19	0.32	52
CWE-685	0.60	1.00	0.75	12
CWE-688	0.57	1.00	0.73	12
CWE-761	0.60	0.56	0.58	117
CWE-789	1.00	0.80	0.89	45
CWE-79	0.00	0.00	0.00	0
CWE-835	1.00	1.00	1.00	6
CWE-89	0.00	0.00	0.00	0
SAFE	0.04	0.01	0.02	7500
EXACTITUD			0.24 (24 %)	15000

Figura 3: Resultados obtenidos en la validación del modelo google/gemma-2-2b-it

6.2. Resultados de la validación de microsoft/Phi-3.5-mini-instruct

En la figura 4 se encuentran los resultados de la validación del modelo.

Los resultados evidencian una alta tasa de falsos positivos, en este caso con las debilidades CWE-20, CWE-151, CWE-180, CWE-184, CWE-190, CWE-244, CWE-467.

Respecto a estos falsos positivos resulta reseñable la clasificación de vulnerabilidades en CWE-79, que es la debilidad de *Cross-Site Scripting* (XSS) y, CWE-801, que no es una vulnerabilidad sino una categoría de vulnerabilidades.

Adicionalmente, ningún caso etiquetado como "SAFE" fue clasificado debidamente, de forma que el modelo clasificó todos los casos recibidos como vulnerables.

A pesar de estos resultados, el modelo microsoft/Phi-3.5-mini-instruct hizo un trabajo mucho mejor que el anterior, puntuando con un 50 % de aciertos. Aunque con un 50 % de aciertos no supera los resultados de Splint, por lo que es peor para la tarea de análisis de vulnerabilidades en código en comparación con el analizador de código estático.

Debilidad	Precisión	Exhaustividad	Puntuación F1	Soporte
CWE-121	0.47	1.00	0.64	1956
CWE-122	0.52	1.00	0.68	1286
CWE-123	1.00	1.00	1.00	8
CWE-124	0.45	1.00	0.62	729
CWE-126	0.46	1.00	0.63	543
CWE-127	0.47	1.00	0.64	777
CWE-151	0.00	0.00	0.00	0
CWE-180	0.00	0.00	0.00	0
CWE-184	0.00	0.00	0.00	0
CWE-190	0.00	0.00	0.00	0
CWE-20	0.00	0.00	0.00	0
CWE-244	0.00	0.00	0.00	0
CWE-401	0.56	1.00	0.72	837
CWE-415	0.59	0.99	0.74	254
CWE-416	0.24	1.00	0.39	29
CWE-467	0.00	0.00	0.00	0
CWE-476	0.63	1.00	0.78	347
CWE-562	0.00	0.00	0.00	2
CWE-590	0.52	1.00	0.68	486
CWE-674	0.00	0.00	0.00	2
CWE-680	0.52	1.00	0.68	52
CWE-685	0.60	1.00	0.75	12
CWE-688	0.48	1.00	0.65	12
CWE-761	0.72	1.00	0.84	117
CWE-789	1.00	1.00	1.00	45
CWE-79	0.00	0.00	0.00	0
CWE-801	0.00	0.00	0.00	0
CWE-835	1.00	1.00	1.00	6
SAFE	0.00	0.00	0.00	7500
EXACTITUD			0.50 (50 %)	15000

Figura 4: Resultados obtenidos 35 en la validación del modelo microsoft/Phi-3.5-mini-instruct

6.3. Procedimiento de ajuste fino de parámetros

Realizamos un pre-procesado de los datos empleados en el ajuste fino de parámetros de forma que se presentasen en formato chat y como instrucciones para las cuales el modelo entrenado diese una respuesta concreta.

La estructura de los datos de entrenamiento son arreglos de tres objetos en los que se indica el texto y el rol desde el cual se interpreta:

- Rol SYSTEM. A partir de este rol se proporciona la instrucción global en la que se define la salvaguarda (*guardrail*). Se especifica el formato de la salida, que ha de ser o bien CWE-XXX, donde XXX es un código de al menos un dígito y de un máximo de 3 dígitos, o el literal "SAFE".
- Rol USER. Se proporciona con este rol el código en lenguaje C a analizar e indicar si contiene o no una vulnerabilidad.
- Rol ASSISTANT. Se proporciona la etiqueta del caso proporcionado.

Esta estructura es la empleada para un proceso de ajuste fino de parámetros, en este caso de una adaptación de bajo rango. Es un proceso de entrenamiento supervisado a partir del cual el modelo construye una matriz de bajo rango a partir de la cual aprender a identificar los fragmentos de código que se le proporcionan para indicar si contiene o no una vulnerabilidad.

Por otro lado, los datos de validación constan de un arreglo con un único objeto, en el cual el texto se proporciona con el rol USER. Esta estructura la empleamos al tratar de emular un *prompt* conversacional como lo haría un usuario interactuando con el modelo.

6.4. Ajuste fino de parámetros y validación del modelo

tiiuae/Falcon3-1B-Instruct

Los modelos Falcon3 son parte de una familia de modelos de inteligencia artificial diseñados y entrenados por el *Technology Innovation Institute* (TII) de los Emiratos Árabes Unidos. El modelo más liviano, de más de mil millones de parámetros, se entrenó en un corpus de texto en los idiomas inglés, francés, español y portugués. El post-entrenamiento se realizó sobre un corpus de 80 mil millones de tokens (o gigatokens) compuesto por páginas web,

problemas o extractos de la ciencia, tecnología, ingeniería y matemáticas (STEM), ejemplos conversacionales y llamadas de funciones de datos [36].

Este modelo en concreto cuenta con una ventana de contexto de hasta 8.000 tókenes, por lo que algunos ejemplos proporcionados en nuestro ajuste fino de parámetros se truncaron hasta la capacidad límite, dado que algunos exceden esa cantidad.

Los resultados de la validación del modelo tras realizar nuestra adaptación de bajo rango pueden observarse en la figura 5. Realizando un análisis de los resultados extraemos las siguientes conclusiones:

Múltiples casos se clasifican con la etiqueta "UNKNOWN". Esta etiqueta se asigna cuando la respuesta proporcionada por el modelo no contiene ni un código de debilidad CWE-XXX con esa estructura, o en su defecto el literal "SAFE". Por lo tanto, una respuesta la cual no se ajusta a ninguna de las anteriores descripciones se clasifica como **desconocida**. En total, 755 casos se clasificaron dentro de esta categoría, de los cuales todos ellos se etiquetaban como "SAFE". Dicho de otra forma, para un 30,2 % de casos no vulnerables, el modelo no supo dar una respuesta válida.

Observamos que para la etiqueta "SAFE" su precisión es perfecta, pero su exhaustividad es realmente baja, siendo de un 15 %. Lo que significa que el modelo generó una gran cantidad de **falsos negativos**.

A diferencia de las dos anteriores validaciones, el modelo no genera una gran cantidad de falsos positivos en etiquetas que no forman parte del conjunto de datos, como sí genera falsos negativos. Por lo tanto, el rendimiento del modelo tras la adaptación de bajo rango es pobre para la tarea realizada, comparándolo además con los resultados proporcionados por Splint.

Debilidad	Precisión	Exhaustividad	Puntuación F1	Soporte
CWE-120	0.00	0.00	0.00	0
CWE-121	0.91	0.40	0.56	660
CWE-122	0.88	0.47	0.61	506
CWE-123	0.67	1.00	0.80	2
CWE-124	0.69	1.00	0.82	251
CWE-126	0.83	0.85	0.84	202
CWE-127	0.87	0.88	0.88	251
CWE-15	0.00	0.00	0.00	0
CWE-401	0.74	0.99	0.85	220
CWE-415	0.91	0.98	0.95	64
CWE-416	0.60	1.00	0.75	18
CWE-476	0.82	1.00	0.90	67
CWE-562	0.00	0.00	0.00	2
CWE-590	0.86	0.93	0.89	168
CWE-68	0.00	0.00	0.00	0
CWE-680	0.92	0.96	0.94	23
CWE-685	0.67	1.00	0.80	6
CWE-688	1.00	1.00	1.00	5
CWE-761	0.55	0.97	0.70	37
CWE-78	0.00	0.00	0.00	0
CWE-789	0.31	0.65	0.42	17
CWE-835	1.00	1.00	1.00	1
CWE-89	0.00	0.00	0.00	0
SAFE	1.00	0.15	0.27	2500
UNKNOWN	0.00	0.00	0.00	0
EXACTITUD			0.43 (43 %)	5000

Figura 5: Resultados obtenidos en la validación del modelo tiiuae/Falcon3-1B-Instruct

6.5. Ajuste fino de parámetros y validación del modelo

meta-llama/Llama-3.2-1B-Instruct

Llama 3.2 es una familia de modelos extensos de lenguaje desarrollados por la empresa Meta Platforms. Este caso en concreto es un modelo ajustado al seguimiento de instrucciones con un tamaño de mil millones de parámetros. El corpus de texto con el que se pre-entrenó está compuesto de 8 idiomas: inglés, francés, alemán, italiano, español, portugués y tailandés. Este modelo está diseñado para la tarea de asistente conversacional o agente de extracción de conocimiento, resumen y reescritura de entradas o *prompts*. Es importante resaltar que los modelos diseñados por Meta Platforms no son de uso libre, requieren de la aceptación de su política de uso previamente a su disposición [37].

Los resultados de la validación del modelo tras realizar nuestra adaptación de bajo rango pueden observarse en la figura 6. A partir de estos resultados extraemos estas conclusiones:

Cuenta con una baja cantidad de falsos positivos de etiquetas que no se recogen en el conjunto de datos. Además, su tasa de falsos positivos para casos "SAFE" es bastante baja.

Su tasa de falsos negativos es realmente baja en comparación con el modelo `tiuae/Falcon3-1B-Instruct`.

Su rendimiento es sobresaliente, rebasando al resto de modelos y a Splint en la realización de la tarea.

Debilidad	Precisión	Exhaustividad	Puntuación F1	Soporte
CWE-121	0.74	1.00	0.85	660
CWE-122	0.84	1.00	0.91	506
CWE-123	0.67	1.00	0.80	2
CWE-124	0.91	1.00	0.95	251
CWE-126	0.97	0.97	0.97	202
CWE-127	0.90	1.00	0.95	251
CWE-129	0.00	0.00	0.00	0
CWE-244	0.00	0.00	0.00	0
CWE-401	0.93	1.00	0.96	220
CWE-415	0.96	1.00	0.98	64
CWE-416	0.53	1.00	0.69	18
CWE-476	0.94	0.97	0.96	67
CWE-480	0.00	0.00	0.00	0
CWE-562	1.00	1.00	1.00	2
CWE-590	0.89	1.00	0.94	168
CWE-680	0.96	1.00	0.98	23
CWE-685	0.86	1.00	0.92	6
CWE-688	1.00	1.00	1.00	5
CWE-761	0.84	1.00	0.91	37
CWE-789	0.65	1.00	0.79	17
CWE-835	1.00	1.00	1.00	1
SAFE	1.00	0.81	0.89	2500
EXACTITUD			0.90 (90 %)	5000

Figura 6: Resultados obtenidos en la validación del modelo meta-llama/Llama-3.2-1B-Instruct

6.6. Ajuste fino de parámetros y validación del modelo

HuggingFaceTB/SmolLM2-1.7B-Instruct

La familia SmolLM2 es una familia de modelos extensos de lenguaje desarrollados y entrenados por Hugging Face Smol Models Research. Respecto a la familia SmolLM1, estos modelos se entrenaron para ejecución de instrucciones, «recuperación» de conocimiento, razonamiento y problemas matemáticos. Concretamente, la variante de mil setecientos millones de parámetros se entrenó en un corpus de un total de 11 billones de tokens, los cuales corresponden a páginas web educativas, conjuntos de datos matemáticos y con código fuente, etc [38].

En pocas palabras, es un modelo orientado al razonamiento con propósito de resolver problemas educativos, y no especialmente para razonar sobre código.

Los resultados de la validación del modelo tras realizar nuestra adaptación de bajo rango pueden observarse en la figura 7. A la vista de estos resultados podemos sacar las siguientes conclusiones:

A diferencia de otras validaciones, no tenemos tantas clases que no se incluyen en el conjunto de datos, solo CWE-120 y CWE-128, las cuales en conjunto hacen menos de un 3 % de las predicciones. Sin embargo, el modelo tiene un rendimiento nulo en el reporte de casos seguros. De hecho, no reporta ningún caso como seguro, identificando todos los casos como maliciosos.

Además, el modelo clasifica incorrectamente múltiples casos. A partir de este hecho podemos indicar que el modelo aprendió el conjunto de datos, pero no aprendió ni a distinguir los casos seguros de los maliciosos ni a distinguir entre un tipo de vulnerabilidad de otro eficazmente.

Debilidad	Precisión	Exhaustividad	Puntuación F1	Soporte
CWE-120	0.00	0.00	0.00	0
CWE-121	0.47	1.00	0.64	660
CWE-122	0.53	1.00	0.69	506
CWE-123	0.22	1.00	0.36	2
CWE-124	0.50	1.00	0.66	251
CWE-126	0.49	1.00	0.66	202
CWE-127	0.51	1.00	0.67	251
CWE-128	0.00	0.00	0.00	0
CWE-401	0.55	0.95	0.69	220
CWE-415	0.61	0.98	0.75	64
CWE-416	0.39	1.00	0.56	18
CWE-476	0.55	0.99	0.71	67
CWE-562	0.00	0.00	0.00	2
CWE-590	0.57	0.91	0.70	168
CWE-680	0.47	0.96	0.63	23
CWE-685	0.00	0.00	0.00	6
CWE-688	0.67	0.40	0.50	5
CWE-761	0.77	0.54	0.63	37
CWE-789	0.94	0.94	0.94	17
CWE-835	1.00	1.00	1.00	1
SAFE	0.00	0.00	0.00	2500
EXACTITUD			0.49 (49 %)	5000

Figura 7: Resultados obtenidos en la validación del modelo HuggingFaceTB/SmolLM2-1.7B-Instruct

Conclusiones

La exposición de todos los resultados obtenidos, tanto de la validación de Splint como de los modelos extensos de lenguaje sobre los que se realizó un ajuste fino de parámetros y de igual forma de los que no, nos conduce a afirmar lo siguiente.

Primero, que herramientas como los analizadores de código estático continúan como una pieza central en el análisis de *code smells*, fallas en el código, debilidades y vulnerabilidades. A pesar de la elevada tasa de falsos positivos reportada por la herramienta Splint, esta herramienta tuvo su última actualización a la versión 3.1.2 en el 12 de julio del año 2007, dicho de otra forma, lleva casi dos décadas sin actualizarse. Es por ello que para un análisis exhaustivo de código escrito tanto en el lenguaje C como C++ se opta por otros analizadores de código estático, como Clang-Tidy o Frama-C-Eva, a los cuales se les da soporte en la actualidad. Por lo tanto, los SAST continúan como herramientas imprescindibles en el control de vulnerabilidades y como comprobadores en tuberías de integración continua y despliegue continuo.

Finalmente, que los modelos extensos de lenguaje y su aplicación en agentes y asistentes se posicionan como herramientas realmente útiles las cuales acompañan a los SAST en cuanto a la detección de bugs, vulnerabilidades, errores que pueden surgir en tiempo de ejecución e inconsistencias de diseño. La principal ventaja que presentan los *LLM* es que su ventana de contexto, en comparación a los modelos que analizamos en esta exposición, los modelos que conforman la frontera del estado del arte cuentan con ventanas de contexto que exceden el millón de tokens. Precisamente esta extensión de ventanas de contexto la motivan sus aplicaciones, especialmente la de entornos de desarrollo integrados (IDE) que integran inteligencia artificial, algunos ejemplos de ellos son Cursor, Windsurf, GitHub Copilot y OpenCode, los cuales reciben como entrada todo el contexto de una aplicación que potencialmente puede exceder el medio millón de tokens en aplicaciones de negocio en fases iniciales de desarrollo.

Con respecto a los resultados obtenidos en la validación de los modelos sobre los que se

realizó un ajuste fino de parámetros, las evidencias recopiladas nos llevan a determinar los siguientes aspectos.

- Que los resultados están influenciados en gran medida por la tipología de datos con los que se entrenaron y se validaron. La suite de pruebas Juliet C/C++ es un conjunto de datos conformado por código sintético, por lo que los resultados obtenidos no son indicativos del éxito o fracaso que puedan tener estos modelos con código de producción real, especialmente porque el código de sistemas de producción empleado en la vida real entraña niveles de complejidad mucho más elevados de los que pueden observarse en las muestras empleadas en el proceso de entrenamiento y validación.
- Que el que un modelo rinda peor o mejor no es indicativo de que el modelo esté o no defectuoso, sino que en una enorme medida dependen del contexto adquirido durante el proceso de entrenamiento o ajuste fino para la ejecución de instrucciones. Todos los modelos sobre los que se realizó el estudio contaban con un contexto en su práctica totalidad determinado por el contenido del conjunto de datos con los que se entrenaron. De hecho, que el modelo meta-llama/Llama-3.2-1B-Instruct obtuviese buenos resultados dice mucho acerca del corpus de datos con el que se entrenó en lugar de su adecuación o no al contexto, pues muy probablemente ya contaba en su contexto con una gran cantidad de tokens pertenecientes a código fuente.
- Que los resultados de la validación de los modelos sobre los que se realizó un ajuste fino de parámetros están en gran medida determinados por la cantidad de casos de prueba, los cuales son 15.000, y que el entrenamiento y validación se realizaron sobre medios de cómputo realmente limitados, en concreto sobre una NVIDIA GeForce RTX 2070 Laptop GPU con 8 GB de VRAM GDDR6.

7.1. Líneas futuras

En líneas futuras pueden emplearse conjuntos de datos enriquecidos con ejemplos de código real en múltiples aplicaciones intensivas en el uso de datos, un ejemplo de esto es el conjunto de datos NIST SARD, que es un repositorio con muestras de código de aplicaciones como SQLite, FFmpeg, Wireshark o Gimp; además de ejemplos de código en el lenguaje Java [39].

Por otro lado, pueden emplearse recursos de mayor envergadura para la adaptación de bajo rango y validación de resultados, con los cuales llevar a cabo un procesamiento de datos a un volumen mucho mayor del llevado a cabo en este proyecto. De esta forma, puede orientarse o bien a modelos con una mayor cantidad de parámetros o a conjuntos de datos mucho más extensos si se quiere continuar la investigación en el contexto de modelos de inteligencia artificial livianos.

Referencias

- [1] Robert C. Seacord. *Secure coding in C and C++*. eng. 2nd ed. SEI series in software engineering Secure coding in C and C++. Place of publication not identified: Addison Wesley, 2013. ISBN: 9780132981972.
- [2] Vahid Eftekhari Moghadam et al. “Memory Integrity Techniques for Memory-Unsafe Languages: A Survey”. En: *IEEE Access* 12 (2024), págs. 43201-43221. doi: 10.1109/ACCESS.2024.3380478.
- [3] Cristina Cifuentes et al. “The role of program analysis in security vulnerability detection: Then and now”. En: *Computers & Security* 135 (2023), pág. 103463. ISSN: 0167-4048. doi: <https://doi.org/10.1016/j.cose.2023.103463>. URL: <https://www.sciencedirect.com/science/article/pii/S0167404823003735>.
- [4] Bushra Aloraini et al. “An empirical study of security warnings from static application security testing tools”. En: *Journal of Systems and Software* 158 (2019), pág. 110427. ISSN: 0164-1212. doi: <https://doi.org/10.1016/j.jss.2019.110427>. URL: <https://www.sciencedirect.com/science/article/pii/S0164121219302018>.
- [5] Cal Erickson. *Memory leak detection in embedded Systems — Linux Journal*. 2002. URL: <https://www.linuxjournal.com/article/6059>.
- [6] Ruohao Guo Jenny Chen. *CS 225 — Stack and Heap Memory*. URL: <https://courses.grainger.illinois.edu/cs225/sp2020/resources/stack-heap/>.
- [7] *CWE - CWE-121: Stack-based Buffer Overflow (4.20)*. URL: <https://cwe.mitre.org/data/definitions/121.html>.
- [8] *CWE - CWE-562: Return of Stack Variable Address (4.20)*. URL: <https://cwe.mitre.org/data/definitions/562.html>.
- [9] *CWE - CWE-244: Improper clearing of heap memory before release ('Heap Inspection') (4.20)*. URL: <https://cwe.mitre.org/data/definitions/244.html>.
- [10] *CWE - CWE-401: Missing Release of Memory after Effective Lifetime (4.20)*. URL: <https://cwe.mitre.org/data/definitions/401.html>.

- [11] *CWE - CWE-415: Double free (4.20)*. URL: <https://cwe.mitre.org/data/definitions/415.html>.
- [12] *CWE - CWE-416: Use after free (4.20)*. URL: <https://cwe.mitre.org/data/definitions/416.html>.
- [13] *CWE - CWE-476: NULL Pointer Dereference (4.20)*. URL: <https://cwe.mitre.org/data/definitions/476.html>.
- [14] *CWE - CWE-590: NULL Pointer Dereference (4.20)*. URL: <https://cwe.mitre.org/data/definitions/590.html>.
- [15] *CWE - CWE-761: NULL Pointer Dereference (4.20)*. URL: <https://cwe.mitre.org/data/definitions/761.html>.
- [16] *CWE - CWE-123: Write-What-Where Condition (4.20)*. URL: <https://cwe.mitre.org/data/definitions/123.html>.
- [17] *CWE - CWE-680: Integer overflow to buffer overflow (4.20)*. URL: <https://cwe.mitre.org/data/definitions/680.html>.
- [18] M McDonough. *Large language model — definition, history, & facts — britannica*. 3 de jun. de 2026. URL: <https://www.britannica.com/topic/large-language-model>.
- [19] Cole Stryker. *¿Qué son los grandes modelos de lenguaje (LLM)?* 27 de nov. de 2025. URL: <https://www.ibm.com/es-es/think/topics/large-language-models>.
- [20] Ashish Vaswani et al. “Attention is all you need”. En: (2017). eprint: 1706.0376. URL: <https://arxiv.org/abs/1706.03762>.
- [21] Ivan Belcic y Cole Stryker. *Fine-tuning eficiente en parámetros*. 27 de nov. de 2025. URL: <https://www.ibm.com/es-es/think/topics/parameter-efficient-fine-tuning>.
- [22] Joshua Noble. *Adaptación de bajo rango*. 27 de nov. de 2025. URL: <https://www.ibm.com/es-es/think/topics/lora>.

- [23] Tollef Emil Jørgensen. “Resource-Efficient Language Models: Quantization for fast and accessible inference”. En: *arXiv (Cornell University)* (13 de mayo de 2025). DOI: 10.48550/arxiv.2505.08620. URL: <https://doi.org/10.48550/arxiv.2505.08620>.
- [24] SonarSource. *Guía para desarrolladores de linter*. 11 de abr. de 2024. URL: <https://www.sonarsource.com/es/resources/library/linter/>.
- [25] Darren Holden y Nafiseh Kahani. “Code linting using language models”. En: *arXiv pre-print arXiv:2406.19508* (2024). URL: <https://arxiv.org/abs/2406.19508>.
- [26] Kirill I. Gladkikh y Alexander A. Zakharov. “Comparison of Language Models for Source Code Vulnerability Classification”. En: *2025 International Russian Automation Conference (RusAutoCon)*. 2025, págs. 779-784. DOI: 10.1109/RusAutoCon65989.2025.11177346.
- [27] José Armando Santas Ciavatta et al. “Integration of Large Language Models (LLMs) and Static Analysis for Improving the Efficacy of Security Vulnerability Detection in Source Code”. En: *Computers, Materials and Continua* 86.3 (2026). ISSN: 1546-2218. DOI: <https://doi.org/10.32604/cmc.2025.074566>. URL: <https://www.sciencedirect.com/science/article/pii/S1546221826000603>.
- [28] *Juliet C/C++ 1.3 - NIST Software Assurance Reference Dataset*. URL: <https://samate.nist.gov/SARD/test-suites/112>.
- [29] *huggingface (Hugging Face)*. URL: <https://huggingface.co/huggingface>.
- [30] *LorenzH/juliet_test_suite_c_1_3 · Datasets at Hugging Face*. URL: https://huggingface.co/datasets/LorenzH/juliet%5C_test%5C_suite%5C_c%5C_1%5C_3.
- [31] Gemma Team et al. “Gemma 2: Improving open language models at a practical size”. En: *arXiv (Cornell University)* (31 de jul. de 2024). DOI: 10.48550/arxiv.2408.00118. URL: <http://arxiv.org/abs/2408.00118>.
- [32] *google/gemma-2-2b-it · Hugging Face*. 12 de mar. de 2001. URL: <https://huggingface.co/google/gemma-2-2b-it>.

- [33] Marah Abdin et al. “PHI-3 Technical Report: A highly capable language model locally on your phone”. En: *arXiv (Cornell University)* (22 de abr. de 2024). doi: 10.48550/arxiv.2404.14219. URL: <http://arxiv.org/abs/2404.14219>.
- [34] *microsoft/Phi-3.5-mini-instruct* · *Hugging Face*. 2 de mar. de 2001. URL: <https://huggingface.co/microsoft/Phi-3.5-mini-instruct>.
- [35] Rina Diane Caballar. *¿Qué es Google Gemma? — IBM*. 28 de nov. de 2025. URL: <https://www.ibm.com/es-es/think/topics/google-gemma>.
- [36] *tiiuae/Falcon3-1B-Instruct* · *Hugging Face*. 6 de nov. de 2025. URL: <https://huggingface.co/tiiuae/Falcon3-1B-Instruct>.
- [37] *meta-llama/Llama-3.2-1B* · *Hugging Face*. 6 de dic. de 2024. URL: <https://huggingface.co/meta-llama/Llama-3.2-1B>.
- [38] *HuggingFaceTB/SMOLLM2-1.7B-Instruct* · *Hugging face*. 5 de mayo de 2025. URL: <https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B-Instruct>.
- [39] *Software Assurance Reference Dataset (SARD)* — *NIST*. 1 de mayo de 2026. URL: <https://www.nist.gov/itl/csd/secure-systems-and-applications/samate/software-assurance-reference-dataset-sard>.